

УНИВЕРСИТЕТ ИТМО

Факультет прикладной информатики

Направление подготовки 45.03.04 Интеллектуальные системы в гуманитарной
сфере

Дисциплина «Мобильная разработка (Android и iOS)»



ОТЧЕТ по
лабораторным работам

Выполнил:
Могильный М.И.

Группа:
К3442
Поток: 2.1

Преподаватель:
Шатинский Г.С.

Санкт-Петербург, 2025 г.

СОДЕРЖАНИЕ

2. Общая архитектура приложения.....	4
2.1. Выбор архитектурного подхода.....	4
2.2. Структура проекта.....	4
3. Навигация и организация экранов.....	4
3.1. Главная Activity.....	4
3.2. Bottom Navigation.....	4
3.3. Navigation Graph.....	5
4. Управление жизненным циклом компонентов.....	5
5. Реализация экранов приложения.....	5
5.1. Экран новостной ленты.....	5
5.2. Экран профиля пользователя.....	6
5.3. Экран настроек.....	6
6. Работа с данными.....	6
6.1. Получение данных из сети.....	6
6.2. Локальное хранение данных.....	6
6.3. Репозиторий как единая точка доступа.....	7
7. Отображение списка сообщений.....	7
7.1. RecyclerView и адаптер.....	7
7.2. Использование Material Design.....	7
7.3. Реализация лайков.....	7
8. Фоновая синхронизация данных.....	7
8.1. Назначение WorkManager.....	7
8.2. Периодическая синхронизация.....	7
9. Система уведомлений.....	8
9.1. Разрешения на уведомления.....	8
9.2. Отображение уведомлений.....	8

ВВЕДЕНИЕ

В рамках выполнения практических работ было разработано Android-приложение типа «Мессенджер», предназначенное для отображения новостной ленты, профиля пользователя и настроек приложения. Основной целью разработки являлось освоение современных подходов к построению Android-приложений, включая архитектуру MVVM, навигацию между экранами, работу с сетью и локальным хранилищем данных, а также реализацию фоновых задач и уведомлений.

Приложение разрабатывалось с использованием языка Kotlin и стандартных компонентов Android SDK, а также библиотек Jetpack.

2. Общая архитектура приложения

2.1. Выбор архитектурного подхода

На рисунке 1 представлена архитектура приложения, основанная на паттерне MVVM. Пользовательский интерфейс реализован с использованием Fragment, которые взаимодействуют с ViewModel через LiveData. ViewModel обращается к репозиторию, инкапсулирующему работу с источниками данных: сетевым API (Retrofit) и локальной базой данных (Room). Для фоновой синхронизации данных используется WorkManager, по результатам работы которого отображаются системные уведомления.

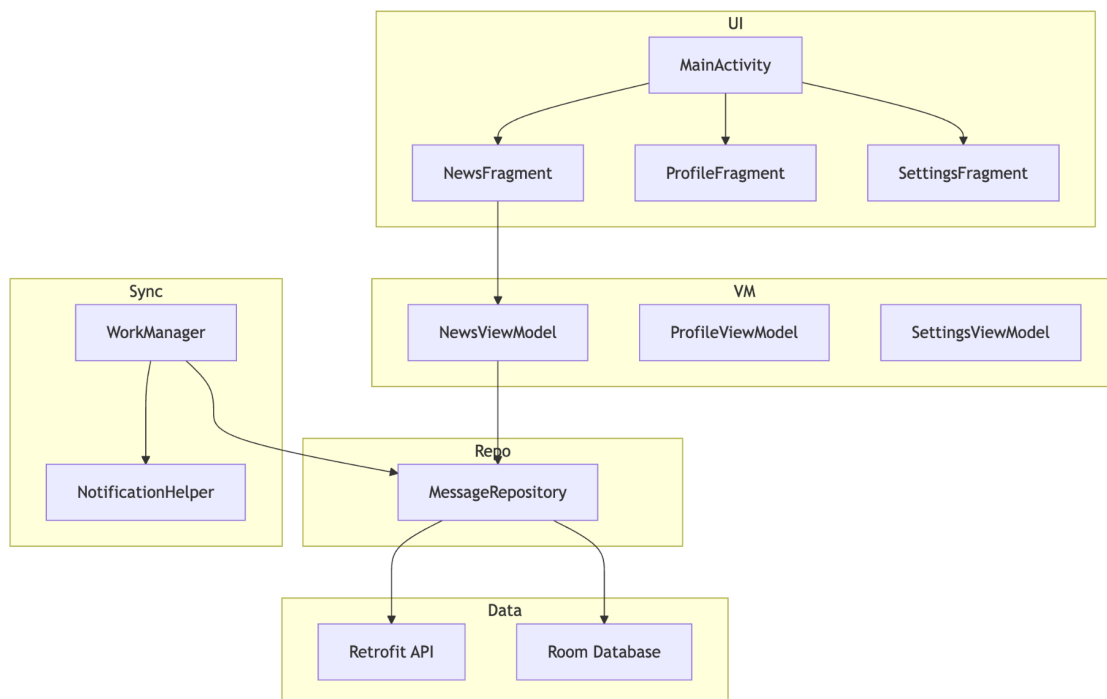


Рисунок 1 - Архитектура MVVM в приложении

Для реализации выбранной архитектуры в проекте используются ViewModel и репозиторий. Пример класса ViewModel, отвечающего за экран новостной ленты, приведен в листинге 1.

Листинг 1 - Пример ViewModel экрана новостей

```
class NewsViewModel(application: Application) :
    AndroidViewModel(application) {

    private val repository = MessageRepository(application)

    private val _messages = MutableLiveData<List<MessageEntity>>>()

    val messages: LiveData<List<MessageEntity>>> = _messages

    fun loadMessages() {

        viewModelScope.launch {

            try {

                Log.d("NewsViewModel", "loadMessages() called")

                val data = repository.getMessage()

                Log.d("NewsViewModel", "Messages loaded: ${data.size}")

                _messages.value = data

            } catch (e: Exception) {

                Log.e("NewsViewModel", "Ошибка загрузки сообщений", e)

            }

        }

    }

}
```

```
fun likeMessage(message: MessageEntity) {  
  
    viewModelScope.launch {  
  
        val updatedMessage = repository.toggleLike(message)  
  
        _messages.value = _messages.value?.map {  
  
            if (it.id == updatedMessage.id) updatedMessage else it  
  
        }  
  
    }  
  
}  
  
}
```

2.2. Структура проекта

Проект организован по функциональным пакетам, включающим:

- 1) UI-компоненты (Activity и Fragment);
- 2) ViewModel для каждого экрана;
- 3) Repository для работы с данными;
- 4) слой работы с сетью и базой данных.

Структура пакетов проекта представлена на рисунке 2.

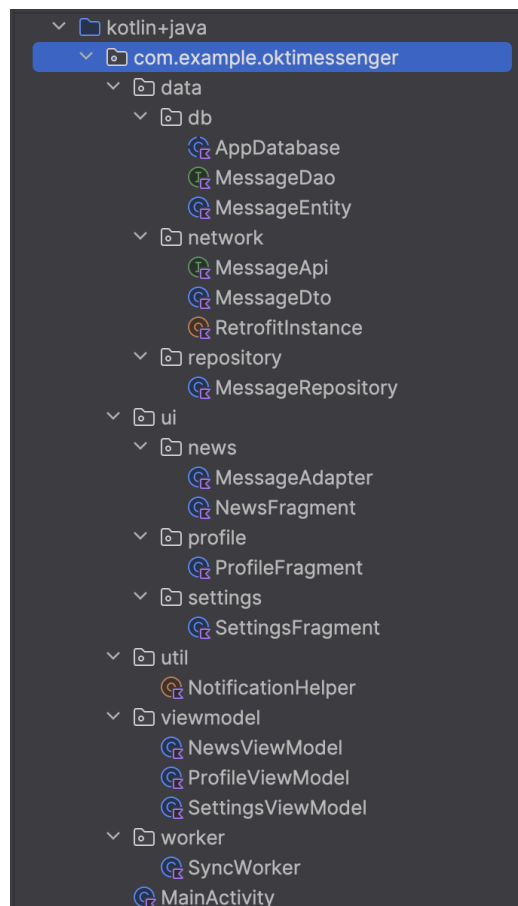


Рисунок 2 - Структура пакетов проекта

3. Навигация и организация экранов

3.1. Главная Activity

В приложении используется одна основная Activity (MainActivity), которая является точкой входа в приложение. Внутри неё размещен контейнер для Fragment, управляемых навигационным компонентом. В листинге 2 представлен фрагмент кода основной Activity, отвечающей за инициализацию навигации и запуск фоновой синхронизации.

Листинг 2 - MainActivity

```
class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private val TAG = "MainActivity"

    override fun onCreate(savedInstanceState: Bundle?) {

        val prefs = getSharedPreferences("settings", MODE_PRIVATE)
        val isDark = prefs.getBoolean("dark_mode", false)
        AppCompatActivity.setDefaultNightMode(
            if (isDark) AppCompatActivity.MODE_NIGHT_YES
            else AppCompatActivity.MODE_NIGHT_NO
        )

        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate")

        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        val navHostFragment = supportFragmentManager
            .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
        val navController = navHostFragment.navController
        binding.bottomNavigationView.setupWithNavController(navController)

        requestNotificationPermission()
        setupPeriodicSync()
    }

    override fun onStart() {
```



```

        super.onStart()
        Log.d(TAG, "onStart")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy")
    }
}

```

3.2. Bottom Navigation

Для переключения между основными разделами приложения используется `BottomNavigationView`. Он обеспечивает быстрый доступ к экранам новостей, профиля и настроек. Нижняя панель навигации приложения представлена на рисунке 3.

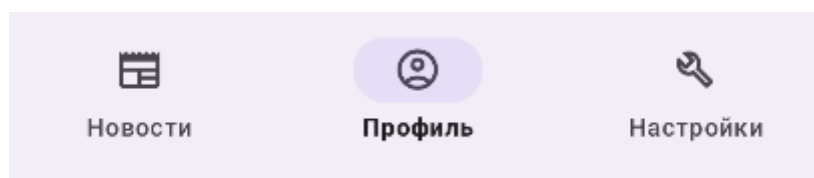


Рисунок 3 - Нижняя панель навигации приложения

3.3. Navigation Graph

Навигация между экранами реализована с помощью Navigation Graph, который описывает все возможные переходы между Fragment. Это позволяет избежать ручного управления транзакциями.

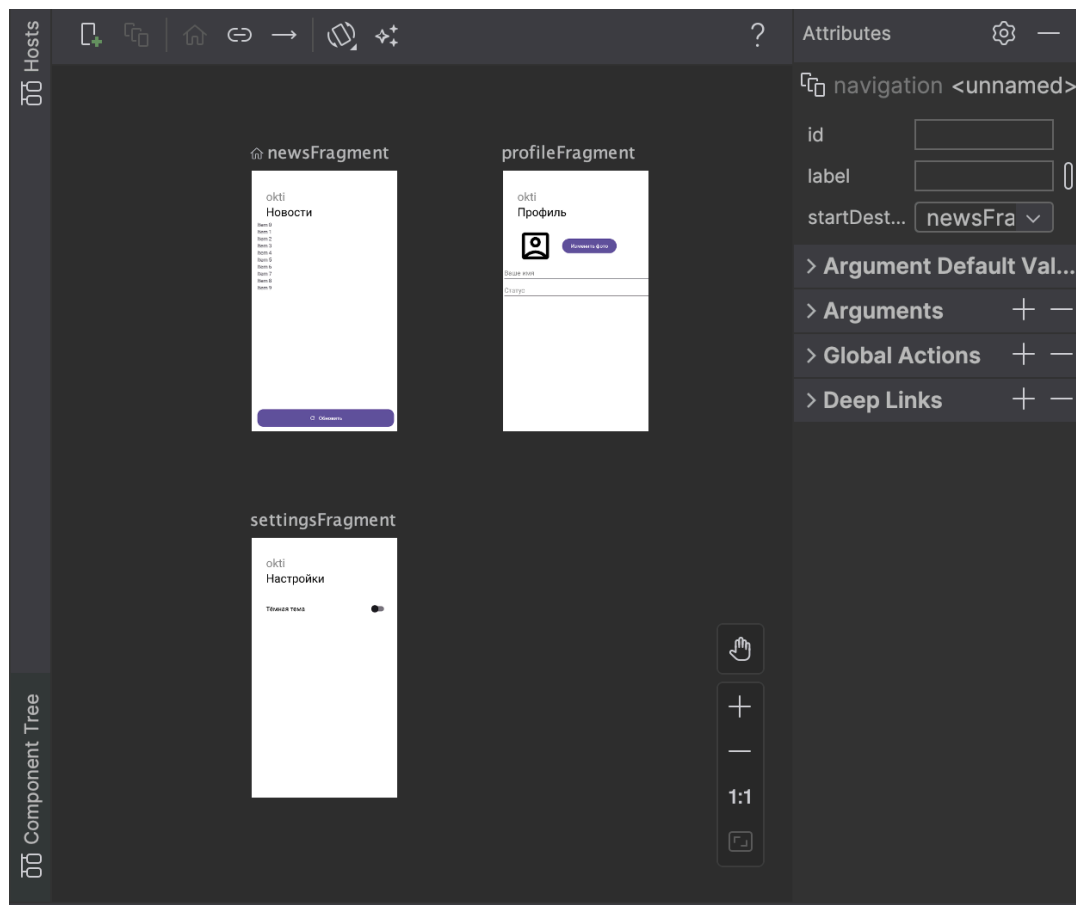
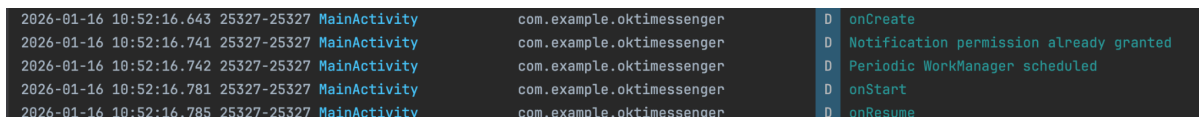


Рисунок 4 - Navigation Graph приложения

4. Управление жизненным циклом компонентов

Для изучения поведения приложения при различных состояниях были добавлены логи жизненного цикла Activity и Fragment. Это позволило убедиться в корректной работе приложения при повороте экрана, сворачивании и закрытии. Примеры логов жизненного цикла компонентов приведены на рисунке 5.



Time	Process	Class	Message
2026-01-16 10:52:16.643	25327-25327	MainActivity	D onCreate
2026-01-16 10:52:16.741	25327-25327	MainActivity	D Notification permission already granted
2026-01-16 10:52:16.742	25327-25327	MainActivity	D Periodic WorkManager scheduled
2026-01-16 10:52:16.781	25327-25327	MainActivity	D onStart
2026-01-16 10:52:16.785	25327-25327	MainActivity	D onResume

Рисунок 5 - Логи жизненного цикла компонентов

Для отслеживания жизненного цикла компонентов использовались сообщения в Logcat. Пример логирования жизненного цикла Fragment приведён в листинге 3.

Листинг 3 - Логирование жизненного цикла Fragment

```
override fun onCreateView(  
    inflater: LayoutInflater,  
    container: ViewGroup?,  
    savedInstanceState: Bundle?  
) : View {  
    Log.d("NewsFragment", "onCreateView called")  
    return inflater.inflate(R.layout.fragment_news, container, false)  
}
```

5. Реализация экранов приложения

5.1. Экран новостной ленты

Экран новостной ленты является центральным экраном приложения и предназначен для отображения списка сообщений, полученных из сети или локальной базы данных. Внешний вид экрана новостной ленты представлен на рисунке 6.

okti

Новости



User 1



quia et suscipit
suscipit recusandae consequuntur
expedita et cum
reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem
eveniet architecto



User 2



est rerum tempore vitae
sequi sint nihil reprehenderit dolor
beatae ea dolores neque
fugiat blanditiis voluptate porro vel nihil
molestiae ut reiciendis
qui aperiam non debitis possimus qui
neque nisi nulla



User 3



et iusto sed quo iure
voluptatem occaecati omnis eligendi aut
ad
voluptatem doloribus vel accusantium
quis pariatur
molestiae porro eius odio et labore et
velit aut



User 4



ullam et saepe reiciendis voluptatem
adipisci

Обновить



Новости



Профиль



Настройки

Рисунок 6 - Экран новостной ленты

5.2. Экран профиля пользователя

Экран профиля позволяет пользователю редактировать персональные данные. Все изменения сохраняются во ViewModel и не теряются при изменении конфигурации устройства. Интерфейс экрана профиля пользователя показан на рисунке 7.

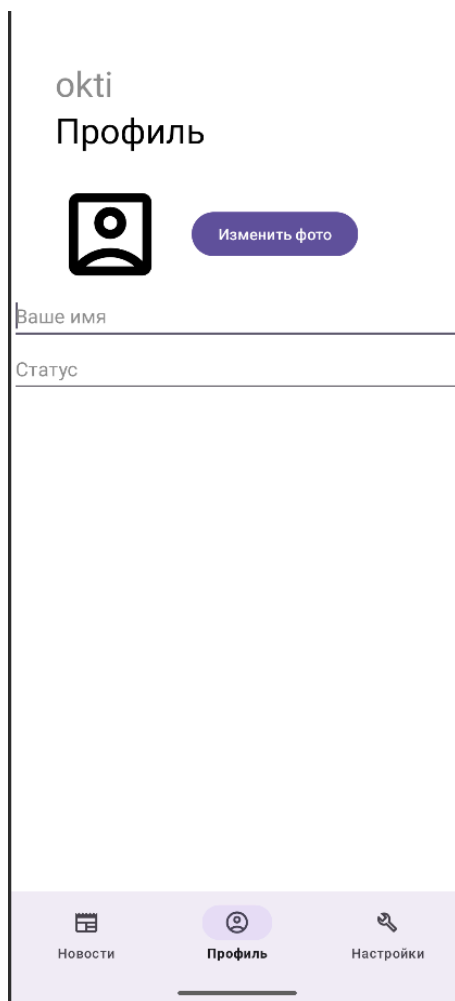


Рисунок 7 - Экран профиля пользователя

5.3. Экран настроек

Экран настроек предоставляет пользователю возможность изменить тему приложения. Выбранное значение сохраняется и применяется при повторном запуске. Экран настроек приложения представлен на рисунке 8.



Рисунок 8 - Экран настроек приложения

6. Работа с данными

6.1. Получение данных из сети

Для получения данных используется библиотека Retrofit в сочетании с корутинами Kotlin. Сетевые операции выполняются асинхронно, не блокируя основной поток.

6.2. Локальное хранение данных

Для реализации локального хранения данных используется библиотека Room. На рисунке 9 представлены сущность базы данных (Entity) и интерфейс доступа к данным (DAO).

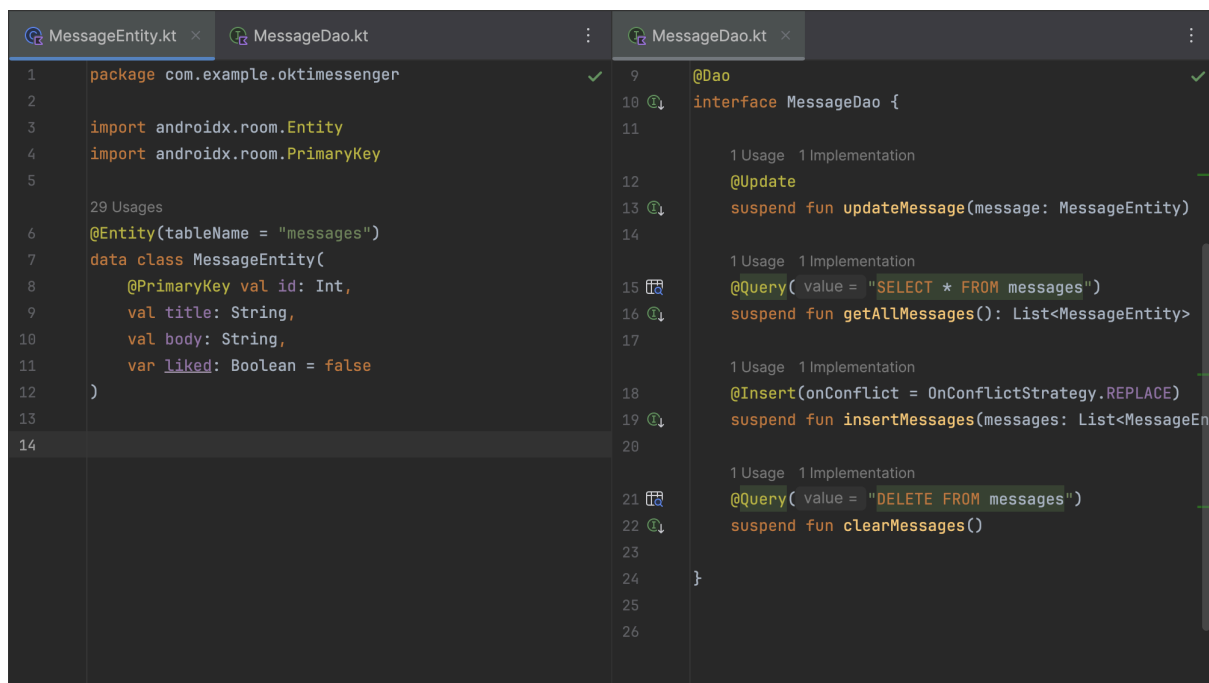


Рисунок 9 - Entity и DAO базы данных

6.3. Репозиторий как единая точка доступа

Repository инкапсулирует логику работы с сетью и базой данных и предоставляет ViewModel единый интерфейс для получения данных.

Листинг 4 - MessageRepository

```
package com.example.oktimessenger.data.repository

import android.content.Context
import com.example.oktimessenger.data.db.AppDatabase
```

```

import com.example.oktimesessenger.data.db.MessageEntity
import com.example.oktimesessenger.data.network.RetrofitInstance

class MessageRepository(context: Context) {

    private val dao =
AppDatabase.Companion.getInstance(context).messageDao()

    private val api = RetrofitInstance.api

    suspend fun getMessages(): List<MessageEntity> {
        return try {
            val response = api.getMessages()
            val entities = response.map {
                MessageEntity(
                    id = it.id,
                    title = it.title,
                    body = it.body
                )
            }
            dao.clearMessages()
            dao.insertMessages(entities)
            entities
        } catch (e: Exception) {
            val cached = dao.getAllMessages()
            cached
        }
    }

    suspend fun toggleLike(message: MessageEntity): MessageEntity {
        val updated = message.copy(liked = !message.liked)
        dao.updateMessage(updated)
        return updated
    }
}

```


7. Отображение списка сообщений

7.1. RecyclerView и адаптер

Для отображения списка сообщений используется RecyclerView с кастомным адаптером, представленным в листинге 5.

Листинг 5 — MessageAdapter

```
package com.example.oktimessenger.ui.news

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView
import com.example.oktimessenger.R
import com.example.oktimessenger.data.db.MessageEntity
import com.example.oktimessenger.databinding.ItemMessageBinding

class MessageAdapter(
    private val onLikeClick: (MessageEntity) -> Unit
) : RecyclerView.Adapter<MessageAdapter.ViewHolder>() {

    private var items = listOf<MessageEntity>()

    fun submitList(list: List<MessageEntity>) {
        items = list
        notifyDataSetChanged()
    }

    inner class ViewHolder(val binding: ItemMessageBinding)
        : RecyclerView.ViewHolder(binding.root)

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
ViewHolder {
        val binding = ItemMessageBinding.inflate(
            LayoutInflater.from(parent.context), parent, false
        )
        return ViewHolder(binding)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val item = items[position]
```

```

        holder.binding.textUser.text = "User ${item.id}"
        holder.binding.textBody.text = item.body

        holder.binding.imageLike.setImageResource(
            if (item.liked) R.drawable.ic_like_on
            else R.drawable.ic_like_off
        )

        holder.binding.imageLike.setOnClickListener {
            onLikeClick(item)
        }
    }

    override fun getItemCount() = items.size
}

```

7.2. Использование Material Design

Интерфейс построен с использованием компонентов Material Design, таких как CardView и FloatingActionButton. Пример оформления карточки сообщения показан на рисунке 10.

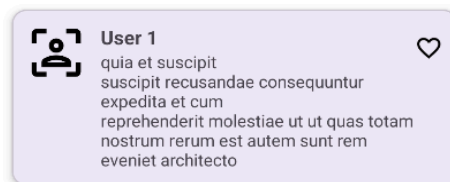


Рисунок 10 - Карточка сообщения

7.3. Реализация лайков

Каждое сообщение можно отметить как понравившееся. Состояние лайка сохраняется и отображается корректно при повторной загрузке данных.

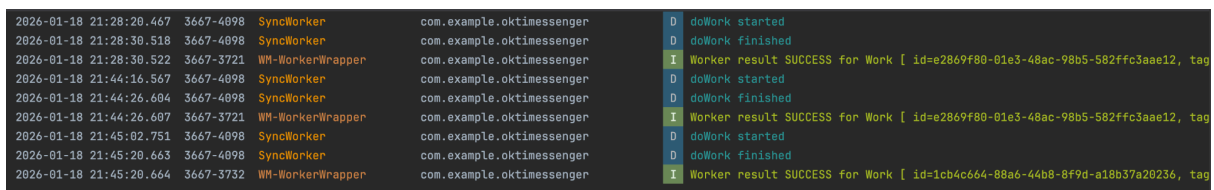
8. Фоновая синхронизация данных

8.1. Назначение WorkManager

Для периодического обновления сообщений используется WorkManager, который гарантирует выполнение задач в фоне.

8.2. Периодическая синхронизация

Синхронизация выполняется через заданные интервалы времени и учитывает состояние сети. Пример выполнения фоновой синхронизации представлен в логах на рисунке 11.



2026-01-18 21:28:20.467	3667-4098	SyncWorker	com.example.oktimesessenger	D doWork started
2026-01-18 21:28:30.518	3667-4098	SyncWorker	com.example.oktimesessenger	D doWork finished
2026-01-18 21:28:30.522	3667-3721	WM-WorkerWrapper	com.example.oktimesessenger	I Worker result SUCCESS for Work [id=e2869f80-01e3-48ac-98b5-582ffc3aae12, tag
2026-01-18 21:44:16.567	3667-4098	SyncWorker	com.example.oktimesessenger	D doWork started
2026-01-18 21:44:26.684	3667-4098	SyncWorker	com.example.oktimesessenger	D doWork finished
2026-01-18 21:44:26.687	3667-3721	WM-WorkerWrapper	com.example.oktimesessenger	I Worker result SUCCESS for Work [id=e2869f80-01e3-48ac-98b5-582ffc3aae12, tag
2026-01-18 21:45:02.751	3667-4098	SyncWorker	com.example.oktimesessenger	D doWork started
2026-01-18 21:45:20.663	3667-4098	SyncWorker	com.example.oktimesessenger	D doWork finished
2026-01-18 21:45:20.664	3667-3732	WM-WorkerWrapper	com.example.oktimesessenger	I Worker result SUCCESS for Work [id=1cb4c664-88a6-44b8-8f9d-a18b37a20236, tag

Рисунок 11 - Логи выполнения WorkManager

Листинг 6 - SyncWorker

```
package com.example.oktimesessenger.worker

import android.content.Context
import android.util.Log
import androidx.work.CoroutineWorker
import androidx.work.WorkerParameters
import com.example.oktimesessenger.util.NotificationHelper
import com.example.oktimesessenger.data.repository.MessageRepository

class SyncWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker(context, params) {

    override suspend fun doWork(): Result {
        Log.d("SyncWorker", "doWork started")

        MessageRepository(applicationContext).getMessages()
        NotificationHelper.show(applicationContext)

        Log.d("SyncWorker", "doWork finished")
        return Result.success()
    }
}
```

9. Система уведомлений

9.1. Разрешения на уведомления

При запуске приложения запрашивается разрешение на отправку уведомлений, необходимое для устройств с Android 13 и выше.

9.2. Отображение уведомлений

Листинг 7 - NotificationHelper

```
package com.example.oktimesessenger.util

import android.Manifest
import android.app.NotificationChannel
import android.app.NotificationManager
import android.content.Context
import android.content.pm.PackageManager
import android.os.Build
import android.util.Log
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat
import com.example.oktimesessenger.R

object NotificationHelper {

    private const val CHANNEL_ID = "sync_channel"
    private const val CHANNEL_NAME = "Синхронизация новостей"

    fun show(context: Context) {

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            if (context.checkSelfPermission(Manifest.permission.POST_NOTIFICATIONS)
                != PackageManager.PERMISSION_GRANTED
            ) {
                Log.d("NotificationHelper", "Нет разрешения на уведомления!")
                return
            }
        }

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val manager =
                context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
            val channel = NotificationChannel(
                CHANNEL_ID,
                CHANNEL_NAME,
                NotificationManager.IMPORTANCE_DEFAULT
            )
            manager.createNotificationChannel(channel)
        }
    }
}
```

```

    )
    manager.createNotificationChannel(channel)
}

val notification = NotificationCompat.Builder(context, CHANNEL_ID)
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("Синхронизация")
    .setContentText("Новые данные получены")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .build()

NotificationManagerCompat.from(context).notify(1, notification)
Log.d("NotificationHelper", "Notification shown")
}
}

```

После успешной синхронизации отображается уведомление с информацией о получении новых данных. Пример системного уведомления, отображаемого после синхронизации данных, показан на рисунке 12.

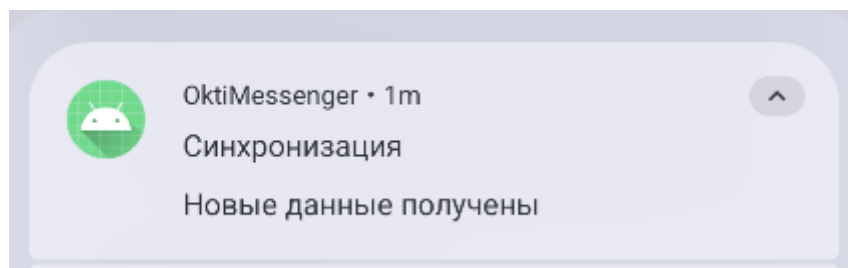


Рисунок 12 - Уведомление о синхронизации

ЗАКЛЮЧЕНИЕ

В результате было разработано полноценное Android-приложение с архитектурой MVVM, поддержкой навигации, загрузкой данных из сети, локальным хранением, фоновыми задачами и системой уведомлений.